

UNIT-3 LAB EXERCISE

NAME: Ranjitha. R

REG NO: 192421285

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N= 8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12

Test Cases :

Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17

The screenshot shows a Python3 IDE with a code editor on the left and a right-hand panel. The code in the editor is as follows:

```
1 n = int(input("Enter number of elements: "))
2 a = list(map(int, input("Enter elements: ").split()))
3
4 minimum = a[0]
5 maximum = a[0]
6
7 for i in range(1, n):
8     if a[i] < minimum:
9         minimum = a[i]
10    if a[i] > maximum:
11        maximum = a[i]
12
13 print("Min =", minimum)
14 print("Max =", maximum)
```

The right-hand panel displays the execution results. At the top, it shows the input: "8" and "5 7 3 4 9 12 6 2". Below this, the "Output" section shows "Status : Successfully executed". Further down, it displays "Time: 0.0100 secs" and "Memory: 8.676 Mb". The "Sample Input" section shows "8" and "5 7 3 4 9 12 6 2". The "Your Output" section shows "Enter number of elements: Enter elements: Min = 2" and "Max = 12".

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to Find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

The screenshot shows a Python3 IDE with a code editor on the left and a right-hand panel. The code in the editor is as follows:

```
1 n = int(input("Enter number of elements: "))
2 a = list(map(int, input("Enter sorted elements: ").split()))
3
4 print("Min =", a[0])
5 print("Max =", a[n - 1])
```

The right-hand panel displays the execution results. At the top, it shows the input: "8" and "2 4 6 8 10 12 14 18". Below this, the "Output" section shows "Status : Successfully executed". Further down, it displays "Time: 0.0100 secs" and "Memory: 9.024 Mb". The "Sample Input" section shows "8" and "2 4 6 8 10 12 14 18". The "Your Output" section shows "Enter number of elements: Enter sorted elements: Min = 2" and "Max = 18".

- You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

```

1 def merge_sort(a):
2     if len(a) > 1:
3         mid = len(a) // 2
4
5         left = a[:mid]
6         right = a[mid:]
7
8         merge_sort(left)
9         merge_sort(right)
10
11         i = j = k = 0
12
13         while i < len(left) and j < len(right):
14             if left[i] < right[j]:
15                 a[k] = left[i]
16                 i += 1
17             else:
18                 a[k] = right[j]
19                 j += 1
20                 k += 1
21
22         while i < len(left):
23             a[k] = left[i]
24             i += 1
25             k += 1
26
27         while j < len(right):
28             a[k] = right[j]
29             j += 1
30             k += 1
31
32 a = list(map(int, input("Enter elements: ").split()))
33 merge_sort(a)
34 print("Sorted array:", *a)

```

31 23 35 27 11 21 15 28

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.048 Mb

Sample Input

31 23 35 27 11 21 15 28

Your Output

Enter elements: Sorted array: 11 15 21 23 27 28 31 35

- Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

```

3 def merge_sort(a):
4     global count
5
6     if len(a) > 1:
7         mid = len(a) // 2
8
9         left = a[:mid]
10        right = a[mid:]
11
12        merge_sort(left)
13        merge_sort(right)
14
15        i = j = k = 0
16
17        while i < len(left) and j < len(right):
18            count += 1
19
20            if left[i] < right[j]:
21                a[k] = left[i]
22                i += 1
23            else:
24                a[k] = right[j]
25                j += 1
26
27            k += 1
28
29        while i < len(left):
30            a[k] = left[i]
31            i += 1
32            k += 1
33
34        while j < len(right):
35            a[k] = right[j]
36            j += 1
37            k += 1
38
39 a = list(map(int, input("Enter elements: ").split()))
40 merge_sort(a)
41 print("Sorted array:", *a)
42 print("Number of comparisons:", count)

```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 8.84 Mb

Sample Input

12 4 78 23 45 67 89 1

Your Output

Enter elements: Sorted array: 1 4 12 23 45 67 78 89
Number of comparisons: 16

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

```

1 def partition(a, low, high):
2     pivot = a[low]
3     i = low + 1
4     j = high
5
6     while True:
7         while i <= high and a[i] <= pivot:
8             i += 1
9
10        while a[j] > pivot:
11            j -= 1
12
13        if i < j:
14            a[i], a[j] = a[j], a[i]
15        else:
16            break
17
18    a[low], a[j] = a[j], a[low]
19    print("After partition:", "a")
20    return j
21
22 def quick_sort(a, low, high):
23     if low < high:
24         p = partition(a, low, high)
25         quick_sort(a, low, p - 1)
26         quick_sort(a, p + 1, high)
27
28 a = list(map(int, input("Enter elements: ").split()))
29 quick_sort(a, 0, len(a) - 1)
30 print("Sorted array:", "a")

```

10 16 8 12 15 6 3 9 5

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.008 Mb

Sample Input

10 16 8 12 15 6 3 9 5

Your Output

Enter elements: After partition: 6 5 8 9 3 10 15 12 16
After partition: 3 5 6 9 8 10 15 12 16
After partition: 3 5 6 9 8 10 15 12 16
After partition: 3 5 6 8 9 10 15 12 16
After partition: 3 5 6 8 9 10 12 15 16
Sorted array: 3 5 6 8 9 10 12 15 16

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

```

1 def quick_sort(a, low, high):
2     if low >= high:
3         return
4
5     i = low
6     j = high
7     pivot = a[(low + high) // 2]
8
9     while i <= j:
10        while a[i] < pivot:
11            i += 1
12
13        while a[j] > pivot:
14            j -= 1
15
16        if i <= j:
17            a[i], a[j] = a[j], a[i]
18            i += 1
19            j -= 1
20
21    print("After partition:", "a")
22
23    if low < j:
24        quick_sort(a, low, j)
25
26    if i < high:
27        quick_sort(a, i, high)
28
29 a = list(map(int, input("Enter elements: ").split()))
30 quick_sort(a, 0, len(a) - 1)
31 print("Sorted array:", "a")

```

19 72 35 46 58 91 22 31

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 8.904 Mb

Sample Input

19 72 35 46 58 91 22 31

Your Output

Enter elements: After partition: 19 31 35 22 58 91 46 72
After partition: 19 22 35 31 58 91 46 72
After partition: 19 22 31 35 58 91 46 72
After partition: 19 22 31 35 58 72 46 91
After partition: 19 22 31 35 46 72 91
After partition: 19 22 31 35 46 58 72 91
Sorted array: 19 22 31 35 46 58 72 91

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

```
1 def binary_search(a, key):
2     low = 0
3     high = len(a) - 1
4     comparisons = 0
5
6     while low <= high:
7         mid = (low + high) // 2
8
9         comparisons += 1
10
11        if a[mid] == key:
12            return mid, comparisons
13        elif key < a[mid]:
14            high = mid - 1
15        else:
16            low = mid + 1
17
18    return -1, comparisons
19
20
21 a = list(map(int, input("Enter sorted elements: ").split()))
22 key = int(input("Enter search key: "))
23
24 index, comparisons = binary_search(a, key)
25
26 if index != -1:
27     print("Position =", index + 1)
28     print("Index =", index)
29 else:
30     print("Element not found")
31
32 print("Comparisons =", comparisons)
```

5 10 15 20 25 30 35 40 45
20

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.108 Mb

Sample Input

5 10 15 20 25 30 35 40 45
20

Your Output

Enter sorted elements: Enter search key: Position = 4
Index = 3
Comparisons = 4

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

```
1 def binary_search(a, key):
2     low = 0
3     high = len(a) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         print("Low =", low + 1, "High =", high + 1,
9               "Mid =", mid + 1, "Value =", a[mid])
10
11        if a[mid] == key:
12            return mid + 1
13
14        elif key < a[mid]:
15            high = mid - 1
16
17        else:
18            low = mid + 1
19
20    return -1
21
22
23 a = list(map(int, input("Enter sorted elements: ").split()))
24 key = int(input("Enter search key: "))
25
26 position = binary_search(a, key)
27
28 if position != -1:
29     print("Element found at position:", position)
30 else:
31     print("Element not found")
```

3 9 14 19 25 31 42 47 53
31

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.3 Mb

Sample Input

3 9 14 19 25 31 42 47 53
31

Your Output

Enter sorted elements: Enter search key: Low = 1 High = 9 Mid = 5 Value = 25
Low = 6 High = 9 Mid = 7 Value = 42
Low = 6 High = 6 Mid = 6 Value = 31
Element found at position: 6

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

```
1 def distance(point):
2     return point[0] ** 2 + point[1] ** 2
3
4
5 points = [[1, 3], [-2, 2], [5, 8], [0, 1]]
6 k = 2
7
8 points.sort(key=distance)
9
10 print("K closest points:", points[:k])
```

3 9 14 19 25 31 42 47 53
31

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.932 Mb

Sample Input

3 9 14 19 25 31 42 47 53
31

Your Output

K closest points: [[0, 1], [-2, 2]]

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples $n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

```
1 from collections import Counter
2
3 A = [1, 2]
4 B = [-2, -1]
5 C = [-1, 2]
6 D = [0, 2]
7
8 count = 0
9
10 AB = Counter()
11
12 for a in A:
13     for b in B:
14         AB[a + b] += 1
15
16 for c in C:
17     for d in D:
18         count += AB[-(c + d)]
19
20 print("Number of tuples:", count)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.084 Mb

Your Output

Number of tuples: 2

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

```
1 def median_of_medians(arr, k):
2
3     if len(arr) <= 5:
4         return sorted(arr)[k - 1]
5
6     groups = []
7
8     for i in range(0, len(arr), 5):
9         group = sorted(arr[i:i + 5])
10        groups.append(group[len(group) // 2])
11
12    pivot = median_of_medians(groups, (len(groups) + 1) // 2)
13
14    smaller = [x for x in arr if x < pivot]
15    equal = [x for x in arr if x == pivot]
16    larger = [x for x in arr if x > pivot]
17
18    if k <= len(smaller):
19        return median_of_medians(smaller, k)
20
21    elif k <= len(smaller) + len(equal):
22        return pivot
23
24    else:
25        return median_of_medians(
26            larger,
27            k - len(smaller) - len(equal)
28        )
29
30
31 arr = [12, 3, 5, 7, 19]
32 k = 2
33
34 print("k-th smallest element:",
35       median_of_medians(arr, k))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0200 secs	8.904 Mb

Your Output

K-th smallest element: 5

12. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

```
1 def median_of_medians(arr, k):
2
3     if len(arr) <= 5:
4         return sorted(arr)[k - 1]
5
6     medians = []
7
8     for i in range(0, len(arr), 5):
9         group = sorted(arr[i:i + 5])
10        medians.append(group[len(group) // 2])
11
12    pivot = median_of_medians(medians, (len(medians) + 1) // 2)
13
14    low = [x for x in arr if x < pivot]
15    equal = [x for x in arr if x == pivot]
16    high = [x for x in arr if x > pivot]
17
18    if k <= len(low):
19        return median_of_medians(low, k)
20
21    elif k <= len(low) + len(equal):
22        return pivot
23
24    else:
25        return median_of_medians(
26            high,
27            k - len(low) - len(equal)
28        )
29
30
31 arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27]
32 k = 5
33
34 result = median_of_medians(arr, k)
35
36 print("K-th smallest element =", result)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.044 Mb

Your Output

K-th smallest element = 21

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.

```
1 from itertools import combinations
2
3
4 def subset_sums(arr):
5     result = []
6
7     for r in range(len(arr) + 1):
8         for combination in combinations(arr, r):
9             result.append((sum(combination), combination))
10
11     return result
12
13
14 def closest_subset(arr, target):
15     mid = len(arr) // 2
16
17     left = arr[:mid]
18     right = arr[mid:]
19
20     left_sums = subset_sums(left)
21     right_sums = subset_sums(right)
22
23     best_sum = 0
24     best_subset = ()
25     best_difference = float('inf')
26
27     for s1, sub1 in left_sums:
28         for s2, sub2 in right_sums:
29             total = s1 + s2
30             difference = abs(target - total)
31
32             if difference < best_difference:
33                 best_difference = difference
34                 best_sum = total
35                 best_subset = sub1 + sub2
36
37     return best_subset, best_sum
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.964 Mb

Your Output

Closest subset: (34, 5, 2)
Subset sum: 41

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

```
1 def subset_sums(arr):
2     sums = set()
3     n = len(arr)
4
5     for mask in range(1 << n):
6         total = 0
7
8         for i in range(n):
9             if mask & (1 << i):
10                 total += arr[i]
11
12         sums.add(total)
13
14     return sums
15
16
17 def meet_in_middle(arr, target):
18     mid = len(arr) // 2
19
20     left = arr[:mid]
21     right = arr[mid:]
22
23     left_sums = subset_sums(left)
24     right_sums = subset_sums(right)
25
26     for s in left_sums:
27         if target - s in right_sums:
28             return True
29
30     return False
31
32
33 arr = [1, 3, 9, 2, 7, 12]
34 target = 15
35
36 print(meet_in_middle(arr, target))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.716 Mb

Your Output

True

15. Given two 2×2 Matrices A and B

$A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 3 & 5 \\ 7 & 5 \end{pmatrix}$

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C = A \times B$.

```
1 def strassen(A, B):
2
3     a, b = A[0]
4     c, d = A[1]
5
6     e, f = B[0]
7     g, h = B[1]
8
9     M1 = (a + d) * (e + h)
10    M2 = (c + d) * e
11    M3 = a * (f - h)
12    M4 = d * (g - e)
13    M5 = (a + b) * h
14    M6 = (c - a) * (e + f)
15    M7 = (b - d) * (g + h)
16
17    C11 = M1 + M4 - M5 + M7
18    C12 = M3 + M5
19    C21 = M2 + M4
20    C22 = M1 - M2 + M3 + M6
21
22    return [[C11, C12],
23            [C21, C22]]
24
25 A = [[1, 7],
26      [3, 5]]
27
28 B = [[6, 8],
29      [4, 2]]
30
31 C = strassen(A, B)
32
33 print("Product Matrix:")
34 for row in C:
35     print(*row)
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.772 Mb

Your Output

Product Matrix:

34 22

38 34

16. Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$

```
1 def karatsuba(x, y):
2
3     if x < 10 or y < 10:
4         return x * y
5
6     n = max(len(str(x)), len(str(y)))
7     m = n // 2
8
9     high1 = x // (10 ** m)
10    low1 = x % (10 ** m)
11
12    high2 = y // (10 ** m)
13    low2 = y % (10 ** m)
14
15    z0 = karatsuba(low1, low2)
16    z1 = karatsuba(low1 + high1, low2 + high2)
17    z2 = karatsuba(high1, high2)
18
19    return (z2 * 10 ** (2 * m)) + \
20           ((z1 - z2 - z0) * 10 ** m) + z0
21
22
23 x = 1234
24 y = 5678
25
26 result = karatsuba(x, y)
27
28 print("Product =", result)
```

Python3

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.056 Mb

Your Output

Product = 7006652